



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-8643 pip entry-point path traversal allows script writes outside the target scripts directory

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-07
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-8643 pip entry-point path traversal allows script writes outside the target scripts directory - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-07 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2026-8643 pip entry-point path traversal allows script writes outside the target scripts directory

1. Executive Summary

CVE-2026-8643 is a path-traversal flaw in `pip`'s wheel-installation workflow. Public CVE, NVD, Python security-announce, GitHub pull-request, and downstream Linux distribution sources agree that a malicious wheel can define crafted `console\_scripts` or `gui\_scripts` names so `pip` writes generated script wrappers outside the intended scripts directory.

The practical risk is best understood as a malicious-package installation problem rather than a remotely reachable network service flaw. An attacker must get a victim to install a crafted wheel, after which the resulting file write is constrained by the permissions of the user running `pip`. That still creates meaningful integrity risk for developer workstations, CI runners, build containers, and package-management workflows that install untrusted or insufficiently vetted Python packages.

2. Vulnerability Metadata

Field	Value
CVE	`CVE-2026-8643`
Product	`pip`
Vulnerable behavior	wheel installation accepted crafted `console_scripts` / `gui_scripts` entry-point names that resolved outside the intended scripts directory
Weaknesses	`CWE-22 Path Traversal`; Red Hat analysis also frames the issue as `CWE-73 External Control of File Name or Path`
CVSS v3.1	`5.5 / MEDIUM`
CVSS vector	`CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N`
CVSS v4.0	`4.1 / MEDIUM`
Affected versions	`pip` versions earlier than `26.1.2` per the CVE Project JSON record
Fixed version	`26.1.2` is present on PyPI and Debian downstream packaging reviewed during this task also treats `26.1.2+dfsg-1` as fixed
Public disclosure date in the CVE record	`2026-06-01`
Reported by	Lumír Balhar

3. Source Review & Confidence

Source	Contribution	Confidence
Reviewed source	Microsoft Security Response Center published the intake item for CVE-2026-8643 and links the issue into the broader vuln	Useful for publication tracking and cross-reference, but not the richest technical source.

Reviewed source	Canonical PSF CNA record states `pip` versions earlier than `26.1.2` are affected; identifies the issue as entry-point p	Highest-confidence source for affected-version boundary and assigner metadata.
Reviewed source	NVD rates the issue `MEDIUM` with `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N` and maps it to `CWE-22`.	High-confidence scoring and weakness source; less detailed than vendor/downstream exploit-context so
Reviewed source	Python security-announce describes CVE-2026-8643 as a medium-severity vulnerability affecting `pip` and points readers t	High-confidence vendor-adjacent disclosure source.
Reviewed source	Upstream PyPA fix titled `Reject entry point names that escape scripts dir` was merged on 2026-05-20; patch logic adds p	Highest-confidence technical remediation source.
Reviewed source	GitHub Advisory tracks the issue as `Moderate` severity and ties it to CVE-2026-8643.	Good corroborating advisory source, though package-version fields were not richly rendered for this
Reviewed source	Red Hat frames the flaw as path traversal via malicious entry-point names in wheel installation, notes arbitrary file ov	High-value source for exploitation preconditions and impact nuance.
Reviewed source	Debian lists vulnerable downstream packaging and records `26.1.2+dfsg-1` as fixed in sid/unstable; notes also reference	Strong downstream packaging corroboration.
Reviewed source	PyPI project metadata confirms both `26.1.1` and `26.1.2` releases exist and shows `26.1.2` as the current version at re	Useful corroboration for release availability and timing.

Primary-source confidence is high on the core issue: Confidence is also high that the issue primarily affects integrity rather than confidentiality or availability. Public material consistently describes arbitrary file overwrite or out-of-directory script installation in the security context of the installing user, but the reviewed sources do not document broad in-the-wild exploitation or a post-overwrite tradecraft pattern that would justify stronger claims.

4. Vulnerability Details

When `pip` installs a wheel, it generates launcher scripts from `console\_scripts` and `gui\_scripts` entry points. The vulnerable logic treated the entry-point name as though it were a safe filename. If a malicious wheel supplied an entry-point name containing path traversal sequences or absolute-path elements, `pip` could resolve that name outside the intended `scheme.scripts` destination and write the generated script wrapper there instead.

The upstream remediation in PyPA pull request `#14000` makes the fix pattern explicit:

- add a helper that normalizes the candidate script path against the target scripts directory
- reject entry-point names that do not remain within that directory after path resolution
- raise an installation error instead of writing the script
- add regression tests covering traversal-style names for both `console\_scripts` and `gui\_scripts`

This makes the vulnerability best characterized as a package-installation path-traversal / arbitrary-file-overwrite issue in wheel entry-point handling, not a flaw in Python package execution after installation.

5. Attack Scenarios

1. A developer installs a malicious wheel from a typo-squatted, compromised, or privately shared package source. During installation, crafted entry-point metadata causes `pip` to write a wrapper script outside the expected scripts directory.

2. A CI/CD job installs unpinned or newly introduced Python dependencies from a package index. A malicious dependency abuses the flaw to overwrite files writable by the build user inside the runner or container filesystem.
3. An internal package repository is poisoned with a crafted wheel that appears legitimate to downstream developers or automation, causing out-of-directory writes during routine install or upgrade operations.

The attacker does not need direct shell access to the victim host, but they do need a delivery path that results in the victim running `pip install` on the malicious wheel.

6. Threat Landscape & Exploitation Status

- Public disclosure exists across CVE, NVD, Python security-announce, Microsoft Security Response Center, GitHub Advisory, Red Hat, and Debian sources.
- The reviewed sources consistently frame this as a medium-severity issue with integrity impact.
- No evidence was collected during this review showing CISA KEV inclusion or confirmed in-the-wild exploitation.
- Exploitation is plausible in supply-chain or social-engineering scenarios because the attacker only needs a victim-controlled install of a crafted wheel, but practical impact is bounded by the permissions of the installing account and the writable paths reachable from that account.

7. Affected Products and Exposure

The strongest supportable statement is that exposure follows `pip` installation workflows, especially where package provenance is weak or automation installs dependencies with broad filesystem access.

Higher-risk patterns:

- Developer workstations that regularly install third-party packages from public indexes or ad hoc wheel files.
- CI/CD runners and ephemeral build environments that perform `pip install` as part of automated dependency resolution.
- Container build stages that run `pip` as root or as a user with write access to sensitive project or runtime paths.
- Internal artifact repositories where package review and signing controls are limited.

Lower-risk patterns:

- Environments already running `pip` `26.1.2` or later.
- Workflows that install only vetted packages from controlled mirrors or with strong provenance checks.
- Virtual environments and sandboxed build contexts that sharply limit what writable paths matter if a malicious wheel is installed.

8. Mitigation and Workarounds

Priority actions:

4. Upgrade `pip` to `26.1.2` or later wherever Python package installation occurs.
5. Treat this issue as a package-provenance problem as well as a patching problem: restrict installs to trusted package sources, approved mirrors, or signed/verified internal artifacts where possible.
6. Avoid running `pip install` with unnecessary privileges, especially in build containers or administrative shells.
7. Prefer isolated virtual environments for development and CI tasks so any attempted overwrite is confined to a narrower filesystem scope.
8. Review automation that accepts arbitrary wheel files, direct URL dependencies, or unreviewed package additions.

9. Detection Engineering

Detection is centered on package-installation telemetry and environment hardening rather than network signatures:

Signal	Telemetry Source	Analytic Goal
Inventory systems and pipelines that invoke <code>pip</code> , including the version installed and whether they resolve dependencies from	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages

public or internal indexes.		
Alert on `pip install` activity involving direct wheel files, unusual package sources, or newly introduced internal dependencies in sensitive build pipelines.	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages
Monitor CI runner and developer endpoint telemetry for unexpected file writes adjacent to `pip install` execution, especially outside expected virtual-environment or scripts directories.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity
Review package-management logs and shell history on developer and build systems when investigating suspicious filesystem changes.	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages
Flag container images and automation templates that still pin vulnerable `pip` releases.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity

9.1 Detection Query

```
// Build and package provenance pivots for CVE-2026-8643 pip entry-point path traversal allows script
writes outside the target scripts directory
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ProcessCommandLine has_any ("go version", "go env", "pip show", "pip list", "syft", "grype",
"sbox")
| project Timestamp, DeviceName, InitiatingProcessAccountName, FileName, ProcessCommandLine
| order by Timestamp desc
```

9.2 Exposure / Triage Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine
has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

10. Threat Hunting

Suggested hunt questions:

- Which developer endpoints, build servers, or containers are still using `pip` versions earlier than `26.1.2`?
- Where do teams permit direct installation of wheel files or unreviewed dependencies?
- Are there recent `pip install` events followed by unexpected file creation or overwrites outside the normal virtual-environment scripts path?
- Do any CI jobs run package installation with elevated permissions that would widen the overwrite blast radius?
- Which internal package repositories or artifact shares allow broad publish rights without review or provenance controls?

10.1 Hunt Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine
has_any ("panic", "fatal error")
```

```
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

11. MITRE ATT&CK Mapping

No ATT&CK technique is assigned with high confidence for this intake review. The vulnerability is real and operationally relevant, but the reviewed public sources describe a package-installation integrity issue rather than a documented adversary procedure, malware family, or intrusion chain that would justify a precise ATT&CK mapping without over-claiming.

12. Validation / Lab Testing

This intake review validated the report against public advisory and package-source material rather than performing a fresh exploit reproduction. Analyst validation steps completed:

- Queried the CVE Project JSON 5 API for the canonical record and affected-version data.
- Queried NVD for CVSS scoring, status, and weakness mapping.
- Reviewed Python `security-announce` disclosure text.
- Reviewed PyPA pull request `#14000` and patch content to confirm the fix rejects entry-point names that escape the scripts directory.
- Reviewed Red Hat Bugzilla metadata for the exploitation preconditions and arbitrary-file-overwrite framing.
- Reviewed Debian Security Tracker and PyPI release metadata to corroborate `26.1.2` as the fixed release boundary.

No Lost Boys Cyber proof-of-concept execution was performed as part of this draft review.

13. Recommended Actions Summary

Priority	Owner	Action
Critical	Platform / Application Owners	Patch `pip` to `26.1.2` or later across developer, CI, and containerized build environments.
High	Third-Party Risk / Asset Management	Reduce trust in unreviewed wheel files and dependency sources; improve package-provenance controls.
High	Security / Platform Owners	Avoid elevated `pip install` contexts and use isolated virtual environments wherever feasible.
High	Security / Platform Owners	Add visibility into `pip` usage and suspicious file-write activity tied to dependency installation workflows.
High	Security / Platform Owners	Treat this CVE as a medium-severity supply-chain and build-integrity issue that is most dangerous in permissive automation environments.

14. References

9. Microsoft Security Response Center, `CVE-2026-8643`
10. CVE Project JSON 5 record for `CVE-2026-8643`
11. NVD entry for `CVE-2026-8643`
12. Python `security-announce` thread for `CVE-2026-8643`
13. PyPA `pip` pull request `#14000`, `Reject entry point names that escape scripts dir`

14. GitHub Advisory `GHSA-wf93-45jw-7689`
15. Red Hat Bugzilla `2460927`
16. Debian Security Tracker entry for `CVE-2026-8643`
17. PyPI project metadata for `pip` release history